Contrôle Continu Programmation C++ - Année 2024/2025

Durée : 1h30 - Documents papiers autorisés

Attention : Vous veillerez à écrire du code le plus propre possible respectant les règles présentées en cours. Notamment, vous ferez attention à nommer explicitement vos variables, vos fonctions ou vos méthodes, à utiliser à bon escient les références, à utiliser correctement le mot clé « const » dès que nécessaire, …

Exercice 1 :

On souhaite modéliser un système de gestion de produits dans un magasin. Chaque produit est défini par un identifiant (nombre), un nom, un prix unitaire, une quantité en stock (0 par défaut).

Q1 Écrire la déclaration de la classe `Produit`, le code du constructeur et le code des getters.

Q2 Ajouter une méthode `ModifierStock` qui prend une quantité en paramètre (positive pour augmenter le stock ou négative pour le diminuer). La méthode doit vérifier si le stock est suffisant en cas de diminution (quantité négative), modifier le stock si possible et retourner `true`. Si la diminution n'est pas possible, retourner `false` sans modifier le stock.

Ajouter une méthode `AppliquerRemise` qui réduit le prix unitaire du produit d'un certain pourcentage passé en paramètre. La méthode doit vérifier que le pourcentage est compris entre 0 et 100 avant d'appliquer la réduction. Si le pourcentage est invalide, elle ne doit rien modifier.

Q3 Écrire une fonction non-membre `calculerValeurTotale` qui prend un objet `Produit` en paramètre et renvoie la valeur totale en stock (quantité * prix). Cette fonction ne doit pas modifier l'objet.

Écrire une fonction non-membre `mettreEnRuptureDeStock` qui prend un objet `Produit` en paramètre et le met en rupture de stock en modifiant sa quantité à 0. Cette fonction doit modifier l'objet.

Q4 Implémenter la surcharge de l'opérateur == pour comparer deux produits (deux produits sont considérés égaux s'ils ont le même identifiant).

Q5 Écrire la surcharge de l'opérateur << permettant d'afficher toutes les informations d'un produit, sous la forme suivante : ID: 1, Nom: Pain, Quantité: 10, Prix: 2.5€.

Exercice 2 :

On souhaite gérer un réseau social simple dans lequel chaque utilisateur est représenté par un objet de la classe `Utilisateur`. Le programme doit permettre de gérer un tableau d'utilisateurs.

Q1 Créer une classe `Utilisateur` avec les attributs suivants : Identifiant (nombre), nom et âge. Fournir un constructeur qui permettent de créer un objet `Utilisateur` et écrire le code des getters.

Q2 Déclarer un tableau d'objets `Utilisateur` et le remplir avec des objets `Utilisateur` que vous aurez créés. Justifier le type de tableau utilisé. Créer une fonction `display` qui prend en paramètre le tableau et retourne une chaine de caractères comprenant une ligne par objet `Utilisateur`. A titre d'exemple, un `Utilisateur` peut être représenté sous la forme ID : 1, Nom : Alice, Âge : 25 ans.

Q3 Écrire une fonction `ModifierAge` qui prend en paramètre le tableau et met à jour l'âge d'un utilisateur identifié par son identifiant. Si l'utilisateur est trouvé, la fonction met à jour son âge et retourne `true`. Si l'utilisateur n'est pas trouvé, la fonction retourne `false`.

Q4 Écrire une fonction `RechercherParNom` qui prend en paramètre le tableau et un nom et qui retourne un objet `Utilisateur` correspondant au nom passé. Pour des raisons de simplification, on suppose que l'utilisateur avec le nom donné existe dans le tableau et qu'il n'y a pas besoin de gérer le cas où le nom n'existe pas.

Q5 Écrire une fonction `CalculerAgeMoyen` qui prend en paramètre le tableau et calcule puis retourne l'âge moyen des objets `Utilisateur` dans le tableau.